

Journal of Statistical Software

*MMMMMM YYYY, Volume VV, Issue II.* *doi: 10.18637/jss.v000.i00*

# A Transparent Financial Risk Factor Library

**Christoph Frey**
Lancaster University

**Christoph Scheuch**
Tidy Intelligence

**Stefan Voigt**
University of Copenhagen

**Patrick Weiss**
Reykjavík University

**Abstract**

The R package **tidyfinance** provides research infrastructure for the empirical asset pricing data workflow. The package exposes every construction decision as a configurable parameter with a documented default. A unified `download_data()` interface covers WRDS (CRSP, Compustat), Ken French's Data Library, FRED, and further sources; modular functions handle fiscal-year lagging, breakpoint computation, and portfolio aggregation. Users adopt the full pipeline or call individual functions inside their own workflows. We illustrate the package by reproducing a classical asset growth factor end-to-end from raw data, then varying construction choices in a single argument. As a demonstration that the infrastructure scales, we publish a precomputed factor library of 50 sorting variables across 841536 specifications, accessible via an open-source API and a dashboard at factors.tidy-finance.org.

*Keywords*: financial risk factors, reproducible research, asset pricing, API, R.

## 1. Introduction

> **Note**
>
> **Section thesis. tidyfinance** is research infrastructure that makes empirical asset pricing reproducible by exposing every construction decision as a configurable parameter with a documented default.
> Argument flow:
> 1. *Hook.* Factor construction embeds dozens of opaque choices that researchers cannot

vary or replicate systematically.
2. *Gap.* Existing libraries (French Data Library, OSAP, Jensen et al.) ship factor *series*; existing R packages (tidyquant, PerformanceAnalytics, quantmod) work downstream of factor construction. No tool covers the upstream workflow.
3. *Contribution.* A unified `download_data()`, modular variable construction, and configurable portfolio sorting in one R package on CRAN.
4. *Demonstration.* A factor library covering 50 variables and 841536 specifications shows the infrastructure scales. This is a side output, not the main contribution.
5. *Roadmap.* Background, package overview, vignette, factor library, summary.

Scope: **py-tidyfinance** is mentioned once at the end of the introduction and once in the data-access section. It is not a co-equal contribution.

Factor returns drive empirical asset pricing tests, portfolio allocation, and performance attribution. Every series, however, embeds dozens of construction choices: how breakpoints are set, whether portfolios are equal- or value-weighted, how delistings are handled, and which fiscal-year lag is applied. These choices are rarely standardized and almost never published with executable code. Researchers who reproduce or extend a factor series must reconstruct each pipeline from scratch, with no guarantee their choices match those of the original authors.

No widely adopted tool covers the upstream workflow. Existing libraries publish factor *series* without code; existing R packages for quantitative finance work downstream of factor construction, on price and return panels that already exist. The result is a missing layer between raw WRDS data and standard asset pricing tests, a layer where construction choices accumulate silently. Section 2 reviews the closest alternatives in detail.

This paper introduces **tidyfinance**, an R package on CRAN that fills the gap. The package treats the empirical asset pricing data workflow as a sequence of three stages: data acquisition, variable construction, and portfolio sorting. A unified `download_data()` interface covers the most common providers; a small set of composable functions handles lagging, breakpoint computation, and portfolio aggregation. Every construction decision is exposed as a configurable parameter with a documented default. Defaults follow established convention (NYSE breakpoints, value weights, July rebalancing, the Fama-French fiscal-year lag), so a default call reproduces standard practice. Researchers who disagree with any default override it with a single argument.

We illustrate the package by reproducing a classical asset growth factor (Cooper, Gulen, and Schill 2008) end-to-end from raw CRSP and Compustat data, then varying its construction choices in a single argument. As a demonstration that the infrastructure scales, we run the package across all defensible combinations of construction choices to produce a precomputed factor library of 50 sorting variables across 841536 specifications. The library is accessible via an API call and an interactive dashboard at factors.tidy-finance.org. Users who prefer Python access the same interface through the companion package **py-tidyfinance**.

The remainder of the paper proceeds as follows. Section 2 places **tidyfinance** among existing libraries and packages. Section 3 walks through the package design and the three workflow stages. Section 4 illustrates the full pipeline on the asset growth factor. Section 5 documents the precomputed factor library and the API. Section 6 concludes.

## 2. Background and related work

Hou, Xue, and Zhang (2020) replicate 452 cross-sectional anomalies and find that most fail to survive out-of-sample, attributing failures in part to undocumented construction variation across studies. Chen and Zimmermann (2022) respond by publishing code for 212 anomaly portfolios, but their library targets a fixed, documented set rather than a reconfigurable pipeline. Jensen, Kelly, and Pedersen (2023) provide a global factor database without an open construction workflow. **tidyfinance** fills the gap all three leave open: every construction decision is explicit, configurable, and version-controlled, so researchers can trace and vary any assumption systematically.

Among R packages for quantitative finance, **tidyquant** (Dancho and Vaughan 2026), **PerformanceAnalytics** (Peterson and Carl 2026), and **quantmod** (Ryan and Ulrich 2025) support time-series retrieval and portfolio performance analytics. None begins at raw accounting and return data, applies the cleaning and lagging conventions factor construction requires, or exposes construction choices as configurable parameters. **tidyfinance** provides this upstream pipeline with a consistent API across both languages. Hence, it enables researchers to access standardized factor series without rebuilding data workflows in each environment.

## 3. Package overview

> **Note**
>
> **Section thesis.** **tidyfinance** reflects three design principles: (i) the empirical asset pricing workflow decomposes into three stages (data acquisition, variable construction, portfolio sorting); (ii) every construction decision is a configurable parameter with a documented default that follows established convention; (iii) functions are composable, so users can adopt the full pipeline or call individual pieces inside their own workflows.
> Argument flow:
> 1. *Workflow diagram (`fig-workflow-diagram`).* Anchors the three-stage design.
> 2. *Installation.* CRAN release plus development version.
> 3. *Data acquisition.* One entry point (`download_data()`), 30+ dataset types, standard cleaning conventions for CRSP / Compustat / TRACE, unified `date` semantics.
> 4. *Variable construction.* Lagging conventions (look-back window vs. Fama-French) make temporal alignment explicit; helpers (`winsorize`, `trim`, `estimate_model`, `compute_rolling_value`) cover routine operations.
> 5. *Portfolio sorting.* Four composable functions (`compute_breakpoints`, `assign_portfolio`, `compute_portfolio_returns`, `compute_long_short_returns`); the high-level `implement_portfolio_sort()` wraps the full pipeline.
>
> Mention **py-tidyfinance** once when illustrating `download_data()`; do not repeat.

### 3.1. Installation and setup

You can install the released version of **tidyfinance** from CRAN via:

```
install.packages("tidyfinance")
```

To install the development version of **tidyfinance** from GitHub, first install the **pak** package if needed, then install the development version:

```
pak::pak("tidy-finance/r-tidyfinance")
```

### 3.2. Design and workflow

**tidyfinance** reflects three design principles. *First*, the empirical asset pricing workflow decomposes into three stages: data acquisition, variable construction, and portfolio sorting. Each stage involves dozens of small decisions: universe filters, cleaning rules, rebalancing conventions, breakpoint definitions. Figure 1 shows the pipeline and the package functions at each stage. *Second*, every construction decision is exposed as a configurable parameter with a documented default. Defaults follow established convention, so a default call reproduces standard practice. *Third*, the functions are composable. Researchers adopt the full pipeline by chaining them, or call individual pieces inside their own workflows.

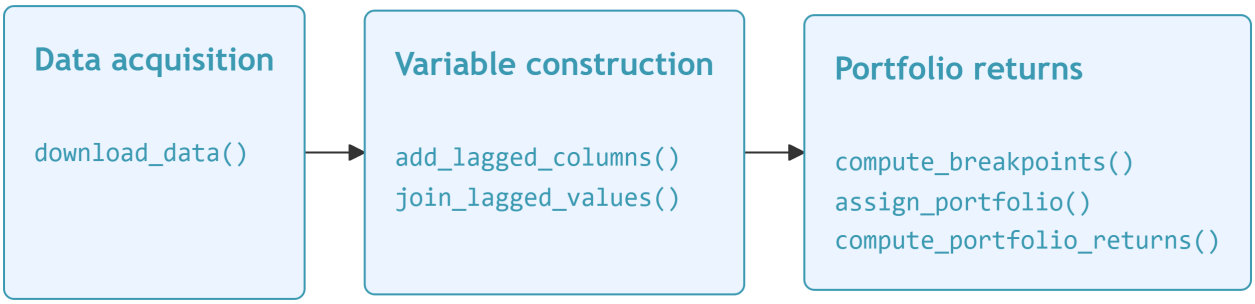



Figure 1: Research workflow for factor construction with **tidyfinance**, illustrating the three-stage pipeline and the package functions involved at each step.

### 3.3. Data access

**tidyfinance** provides a single interface, `download_data()`, to 320 dataset types: equity returns, fixed-income transactions, macroeconomic predictors, index constituents, and more. Table 1 lists all available sources. `list_supported_types()` and `list_supported_indexes()` return the full catalogue programmatically. The companion Python package **py-tidyfinance** exposes the same `download_data()` interface, with identical signatures and dataset identifiers, so analyses can be ported between the two languages without changes to the data layer.

| Source | Description | Example |
|---|---|---|
| **Public data sources** | | |
| Ken French's Data Library | Fama-French 169+ factor models, portfolios, industry classifications | `download_data( "famafrench", "Fama/French 5 Factors")` |

| Source | Description | Example |
|---|---|---|
| Global Q | Hou-Xue-Zhang Q5 factor model (market, size, investment, ROE) at multiple frequencies | `download_data( "globalq", "factors_q5_monthly")` |
| Goyal-Welch Predictors | Macroeconomic return predictors (DP, EP, BM, spreads, etc.) | `download_data( "macro_predictors", "monthly")` |
| FRED | St. Louis Fed economic time series (GDP, CPI, etc.) | `download_data( "fred", series = "GDP")` |
| Yahoo Finance | Daily OHLCV with adjusted close for any ticker | `download_data( "stock_prices", symbols = "AAPL")` |
| Index Constituents | Current members of 21 major indexes (S&P 500, Russell, DAX, MSCI World, etc.) | `download_data( "constituents", index = "DAX")` |
| Open Source Asset Pricing | 213+ monthly asset pricing signals and anomalies | `download_data("osap")` |
| **WRDS (subscription required)** | | |
| CRSP | Stock returns, market cap, exchange, industry | `download_data( "wrds", "crsp_monthly")` |
| Compustat | Annual and quarterly financial statements | `download_data( "wrds", "compustat_annual")` |
| CCM Links | CRSP-Compustat linking table (permno to gvkey) | `download_data( "wrds", "ccm_links")` |
| FISD | Mergent Fixed Income Securities Database | `download_data("wrds", "fisd")` |
| TRACE | Corporate bond transactions (Dick-Nielsen cleaned) | `download_data( "wrds", "trace_enhanced", cusips = "655844CJ5")` |

Table 1: Data sources available via `download_data()` in **tidyfinance**.

`download_data()` applies standard cleaning conventions automatically. For CRSP monthly data (version 2), the function restricts the universe to common equity securities traded on NYSE, AMEX, or NASDAQ. It computes market capitalisation as shares outstanding times price, constructs a one-month lag for portfolio weights, and maps exchange codes and SIC codes to readable labels. Excess returns are raw returns minus the Fama-French risk-free rate; observations with missing returns or market capitalisation are dropped. CRSP version 2 incorporates delisting returns directly, eliminating the need for a separate adjustment.

For Compustat annual data, `download_data()` restricts to industrial-format consolidated statements and retains only the most recent observation per firm-year. It constructs book

equity through a coalesce hierarchy: stockholders' equity, then common equity plus preferred stock, then total assets minus total liabilities. The function adjusts for deferred taxes and investment tax credits and subtracts preferred stock.

For TRACE corporate bond transactions, `download_data()` applies the Dick-Nielsen cleaning protocol (**?**) to remove cancellations, corrections, and reversals before returning the trade panel.

All datasets returned by `download_data()` use a single `date` column that follows the same rules across frequencies. Daily observations return the trading date. Monthly observations are dated to the last calendar day of the reference month, regardless of whether that day was a trading day. Quarterly and annual observations take the fiscal period-end date as reported by the original source; for Compustat, this is the firm's fiscal year-end (`datadate`), which can differ from December across firms and across years for the same firm. Macroeconomic series from FRED keep their original release dates. A simple equality join on `date` therefore aligns observations of the same frequency, while cross-frequency joins, such as merging monthly returns with annual fundamentals, require the lagging logic described in the next subsection.

### 3.4. Variable construction

Most predictive variables in asset pricing studies are based on lagged information so that portfolio formation in month $t$ should always use only data observable at the end of month $t-1$. `add_lagged_columns()` automates this. For each row in the data, it looks back over a configurable window and attaches the most recent observed value of the chosen columns. By default (`ff_adjustment = FALSE`), the window is bounded by `lag` (the minimum delay between observation and use) and `max_lag` (the maximum delay before a stale value is treated as missing). Within this window, the function picks the most recent non-missing value; if no observation falls in the window, the lagged column takes `NA`.

A second mode, enabled by `ff_adjustment = TRUE`, implements the Fama-French convention (Fama and French 1993). Accounting variables for fiscal year $t-1$ are assumed to become public information by June of year $t$ and are then used for portfolio formation from July of $t$ through June of $t+1$. This produces step-function lagged values that update once per year at the June/July boundary, regardless of the underlying fiscal year-end. Figure 2 contrasts the two mechanisms. The upper panel shows a standard look-back window controlled by `lag` and `max_lag`. The lower panel implements the Fama-French convention: fiscal-year data from year $t-1$ becomes available only from July of year $t$, so all months from July through June share the same lagged value locked in at the previous rebalancing. In both cases, `add_lagged_columns()` selects the most recent observed value within the window.

`join_lagged_values()` is a related helper that merges lagged values from one data frame into another using the same windowing logic. It is useful when the target panel and the source panel are at different frequencies, for example when joining monthly accounting data into a quarterly target panel. The function takes the same `ff_adjustment` argument as `add_lagged_columns()`, so the Fama-French convention can be applied at the join step rather than as a separate transformation.

A handful of additional helpers cover routine operations that recur across empirical studies. `winsorize()` and `trim()` handle outliers in cross-sectional variables: the former replaces

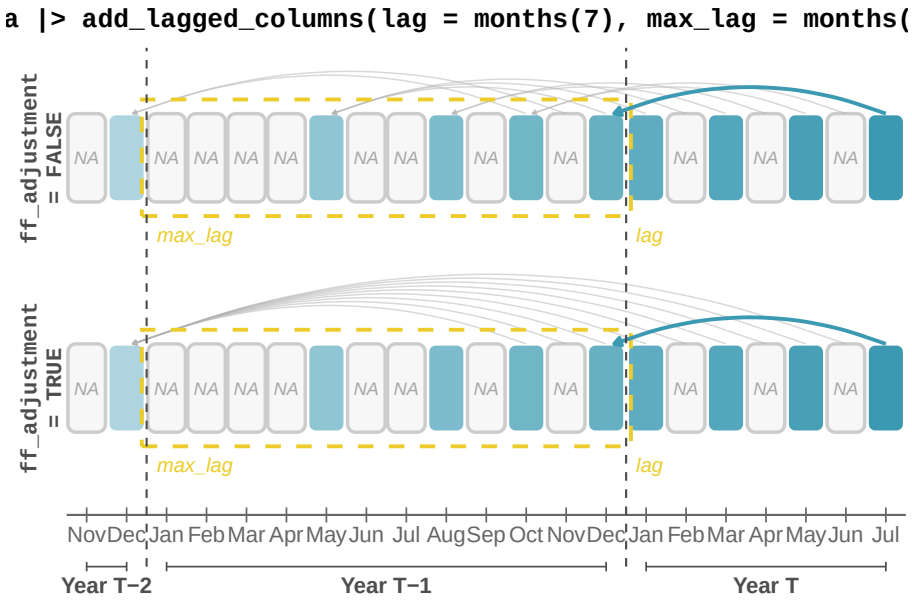



Figure 2: Two lagging mechanisms in `add_lagged_columns()`. Each row represents a data series with observed (filled) and missing (hollow) cells. Dashed rectangles mark the search window; arrows show which value gets matched to each target month.

values beyond user-specified quantiles with the quantile values themselves, while the latter sets them to `NA`. `estimate_model()` fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame, which composes naturally with `group_by()` for grouped or rolling regressions such as time-series factor exposures. `compute_rolling_value()` applies an arbitrary aggregation function over a rolling window defined in calendar time rather than row counts, accommodating irregular observation frequencies and missing periods.

### 3.5. Portfolio sorting

Portfolio sorts are among the most widely used tools in empirical asset pricing (Bali, Engle, and Murray 2016). The procedure groups stocks each period into portfolios based on a sorting variable, such as size, beta, book-to-market, or momentum, and then compares return distributions across the resulting portfolios. The long-short return between the extreme portfolios is interpreted as a factor-mimicking strategy and serves as the basis for tests of whether the sorting variable forecasts future returns. We expose the procedure through four composable functions.

`compute_breakpoints()` takes a cross-section and a sorting variable and returns the numeric cutoffs that define the portfolio boundaries. The user supplies either a number of portfolios (for example, five for quintiles) or an explicit vector of percentiles, but not both. A `breakpoints_exchanges` argument restricts the universe used to compute breakpoints. The standard convention is NYSE-only, which prevents the large cross-section of small NASDAQ firms from dominating the lower bins. A related `breakpoints_min_size_threshold` argument excludes stocks below a market-cap quantile from the breakpoint computation, which is useful

for size-related sorts where microcaps would otherwise distort the cutoffs. A `smooth_bunching` flag handles cases where the sorting variable clusters at the edges by pinning extreme values to the boundary portfolios and recomputing breakpoints on the remainder.

`assign_portfolio()` maps each observation to a portfolio integer using the breakpoints. The breakpoint computation is delegated to a user-supplied function (defaulting to `compute_breakpoints()`), so custom rules can be plugged in without rewriting the assignment logic. In practice, `assign_portfolio()` is called separately for each rebalancing date, so that portfolios are reformed each period rather than fixed once for the whole sample.

`compute_portfolio_returns()` ties the lower-level pieces together into a single call. Given a panel of stock data, one or two sorting variables, a sorting method, and `breakpoint_options`, it handles rebalancing, breakpoint computation, portfolio assignment, and return aggregation. It then returns a complete panel of portfolio returns. For each portfolio-date the function reports three return measures: equal-weighted, value-weighted using `mktcap_lag` as the weight, and capped value-weighted. The capped version pulls weights above the `cap_weight` percentile (default 0.8) back to that cap, which prevents a small number of megacap firms from dominating the portfolio. A `min_portfolio_size` argument sets the minimum number of constituents below which the corresponding return is set to `NA`. An optional `rebalancing_month` argument switches from period-by-period rebalancing to annual rebalancing in the specified month. Portfolios formed in that month are then held constant for the following twelve months.

The function supports three sorting schemes, illustrated in Figure 3. A *univariate* sort uses a single variable. An *independent bivariate* sort computes breakpoints on each variable separately and intersects the resulting bins to produce a two-way grid. A *dependent bivariate* sort first partitions the cross-section by the secondary (control) variable and then re-sorts within each bucket on the main variable, which is the appropriate choice when the second variable is treated as a control rather than as an independent dimension of variation. For bivariate sorts, returns are averaged over the secondary variable so that only the main-variable portfolios are returned.

`compute_long_short_returns()` is a thin convenience layer that takes the output of `compute_portfolio_returns()` and computes the spread between the top and bottom portfolios. A `direction` argument controls the sign and defaults to `"top_minus_bottom"`. The function returns one long-short series per return measure available in the input, so the equal-weighted, value-weighted, and capped value-weighted spreads are produced in a single call. This long-short return is the factor-mimicking spread that the asset-pricing literature typically takes as the object of interest.

For users who want the full pipeline in a single call, `implement_portfolio_sort()` wraps the four functions above. It takes a panel of stock data, one or two sorting variables, a sorting method, and a configuration object, and returns the long-short factor return series alongside the underlying portfolio returns. The lower-level functions remain available for custom workflows that need finer control over breakpoints, weighting, or rebalancing.

## 4. Vignette: A case study in factor construction

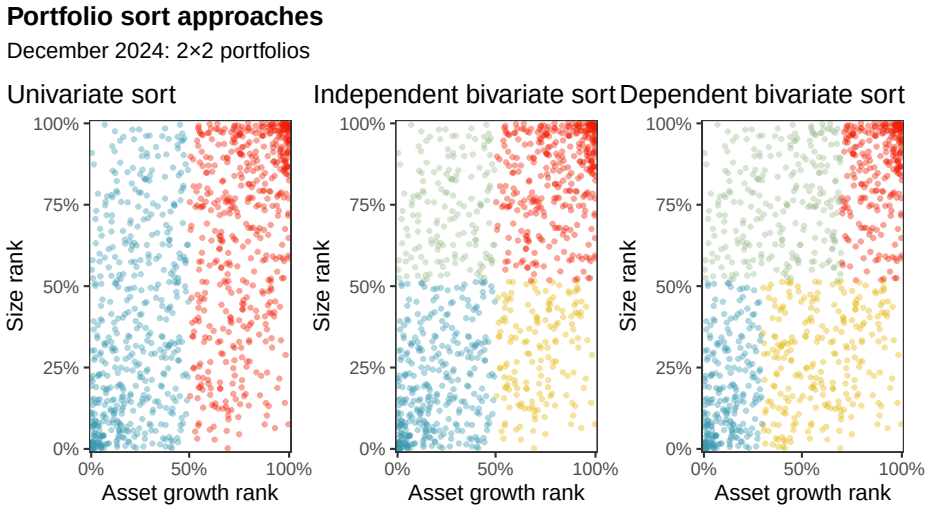



Figure 3: Portfolio sort approaches illustrated for a single month (December 2024). Each panel plots stock-month observations assigned to `r .QuartoInlineRender(n_portfolio)`×`r .QuartoInlineRender(n_portfolio)` portfolios based on asset growth rank (x-axis) and size rank (y-axis), both expressed as percentiles. Colors indicate portfolio cell assignment. The univariate sort (left) assigns portfolios on asset growth alone; the independent bivariate sort (center) assigns each dimension independently; the dependent bivariate sort (right) sorts on size first, then on asset growth within size groups.

> **Note**
>
> **Section thesis.** A worked example shows that **tidyfinance** reproduces the Cooper et al. (2008) asset growth factor end-to-end from raw CRSP and Compustat data, and that varying a construction choice is a one-line change.
> Argument flow:
> 1. *Why asset growth.* Classical, well-replicated factor; the full pipeline is visible (download, lag, join, sort).
> 2. *Default specification.* Value-weighted, NYSE breakpoints, July rebalancing, quintile sort. Reproduces the negative premium documented by Cooper *et al.* (2008).
> 3. *Sensitivity check.* Dependent bivariate sort with size as the second dimension. The premium survives size control. Demonstrates that switching specifications is one argument change.
> 4. *Generalisation.* The same workflow applies to any sorting variable in Table 8.
>
> Do not show every cleaning step (earnings filter, age filter); footnote them and refer to the API for preprocessed panels.

We walk through the construction of the asset growth factor from raw CRSP and Compustat data to long-short portfolio returns, applying each stage of Figure 1 in turn. Each step uses a **tidyfinance** function and generalises directly to any sorting variable in the factor library.[1]

Once installed, load **tidyfinance** and a minimal set of packages for data manipulation and

[1]Installation of **tidyfinance** is described in Section 3.1.

visualisation.

```
library("tidyfinance")

library("dplyr")
library("tidyr")
library("lubridate")
```

### 4.1. Data download

Before the first WRDS download, call `set_wrds_credentials()` to store WRDS credentials securely in `.Renviron`; subsequent `download_data()` calls establish the database connection automatically.

```
date_start <- as.Date("1960-01-01")
date_end <- as.Date("2024-12-31")
```

First, we download monthly stock returns from CRSP. Setting `add_ccm_links = TRUE` appends CRSP-Compustat linking identifiers needed to merge with financial statement data in the next step.

```
crsp_monthly <- download_data(
  domain = "wrds",
  dataset = "crsp_monthly",
  version = "v2",
  start_date = date_start,
  end_date = date_end,
  add_ccm_links = TRUE
)
```

The dataset contains 3451594 stock-month observations for 26263 unique stocks, covering the period from 1960-01-01 to 2024-12-01.

Next, we download annual financial statements from Compustat.

```
compustat_annual <- download_data(
  domain = "wrds",
  dataset = "compustat_annual",
  start_date = date_start,
  end_date = date_end,
  additional_columns = c("at", "ib"),
  only_us = TRUE
)
```

Annual Compustat data contains 350810 firm-year observations for 22615 unique firms, covering the period from 1960-01-01 to 2024-12-01. The `additional_columns` argument retrieves total assets (`at`) and income before extraordinary items (`ib`), needed for computing asset growth and applying an earnings filter in later steps. Setting `only_us = TRUE` restricts the dataset to USD-denominated observations, ensuring consistency with CRSP.

### 4.2. Data cleaning and variable construction

We illustrate data cleaning and variable construction using asset growth (AG) as our running example. Asset growth is the year-over-year percentage change in total assets (Cooper *et al.* 2008),

$$AG_t = (AT_t - AT_{t-1})/AT_{t-1}.$$

The `add_lagged_columns()` function from **tidyfinance** handles temporal alignment by creating a lagged version of total assets grouped by firm identifier (`gvkey`) with a one-year lag. Non-finite values (arising when lagged total assets are zero or missing) become `NA`.[2]

```
sorting_variable_data <- compustat_annual |>
  add_lagged_columns(
    "at",
    lag = years(1),
    by = "gvkey",
  ) |>
  mutate(
    asset_growth = (at - at_lag) / at_lag,
    asset_growth = if_else(is.finite(asset_growth), asset_growth, NA_real_)
  ) |>
  select(gvkey, date, asset_growth, ib)
```

`sorting_variable_data` contains one observation per firm-year, the natural unit for cross-sectional analysis of annual accounting variables. We use `dplyr::filter_out()` (Wickham, François, Henry, Müller, and Vaughan 2026) to drop rows that match a predicate (here, missing asset-growth values) and `tidyfinance::winsorize()` to clip numeric vectors at specified quantile bounds; we apply the latter at the 1st and 99th percentiles before computing summary statistics. Asset growth is heavily right-skewed: a small number of firms with dramatic balance-sheet expansions generate extreme positive values. Table 2 reports the pooled cross-sectional summary; Table 3 shows how those moments vary across calendar years.

| N | Mean | SD | P10 | P25 | Median | P75 | P90 |
|---|---|---|---|---|---|---|---|
| 303671 | 0.235 | 0.786 | -0.181 | -0.028 | 0.071 | 0.216 | 0.596 |

Table 2: Cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). The table reports the number of firm-year observations (N), the mean, standard deviation (SD), and selected percentiles of the annual asset growth distribution computed from `sorting_variable_data`.

[2]For the sake of simplicity we do not show every step of the data cleaning process in the paper. For instance, we apply an earnings filter to exclude firms with negative or zero income before extraordinary items (`ib`), which is a common practice in the literature to mitigate the influence of financial distress on asset growth portfolios. This filter is applied in the code but not shown in the paper for brevity. We also apply an age filter to exclude firms younger than two years, which is common in the literature to ensure that the sorting variable is based on stable financial information. This filter is also applied in the code but not shown in the paper for brevity. The factor library API provides preprocessed panels with all these filters applied, so users can access the final sorting variables without having to repeat these steps themselves.

| Years | Avg N | Mean (avg) | Mean (min) | Mean (max) | SD (avg) | Median (avg) | Median (min) | Median (max) |
|---|---|---|---|---|---|---|---|---|
| 64 | 4745 | 0.239 | 0.042 | 0.904 | 0.788 | 0.072 | 0.001 | 0.134 |

Table 3: Time-series averages of annual cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). For each calendar year, cross-sectional mean, standard deviation, and median are computed from `sorting_variable_data`; the table reports the time-series mean, minimum, and maximum of those annual statistics.

The next step merges annual sorting variables onto the monthly CRSP stock panel using `join_lagged_values()`. The function aligns fiscal-year-end data with monthly observations through a configurable lag window. Setting `ff_adjustment = TRUE` implements the Fama-French convention (Fama and French 1993), aligning fiscal-year data from year $t$ with portfolio formation in July of year $t+1$ through a 12-to-18-month look-back window. The same lagging process generalises to all sorting variables in the full factor library, and the factor library API provides precomputed lagged panels for all of them.

We now apply `join_lagged_values()` to merge the sorting variable onto the monthly return panel. Figure 4 shows the 33rd, 50th, and 66th percentiles of asset growth over time in the resulting panel; pronounced spikes during economic expansions and contractions highlight the cyclical nature of corporate investment that portfolio sorts aim to exploit.

```
sorting_data <- crsp_monthly |>
  join_lagged_values(
    sorting_variable_data,
    "gvkey",
    min_lag = months(7),
    max_lag = months(18),
    ff_adjustment = TRUE
  )

sorting_data <- sorting_data |>

  select(
    date,
    permno,
    exchange,
    siccd,
    ret_excess,
    mktcap_lag,
    asset_growth,
    ib
  ) |>
  filter(date > date_start + months(12) + months(18))
```

The filter removes the initial year used to compute the growth rate plus an additional 18 months to avoid mechanically introduced gaps from the lagging step, leaving a panel of stock-month

observations with excess returns, exchange membership, lagged market capitalisation, the lagged sorting variable. The auxiliary columns `siccd` and `ib` support the sample-construction filters in the customised subsection 4.3.2.

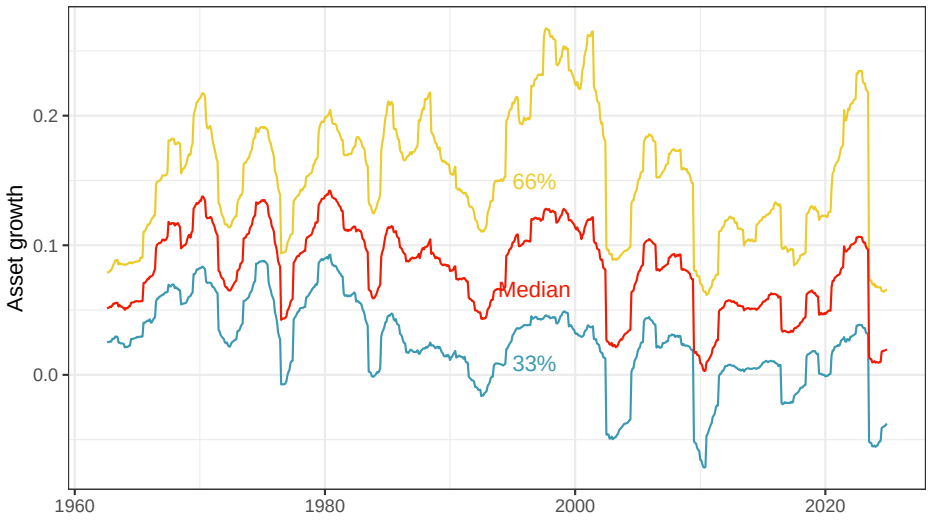



Figure 4: Distribution of asset growth over time. Lines show the 33rd, 50th, and 66th percentiles of the cross-sectional distribution of asset growth in `sorting_data`, the firm-month panel used as input for portfolio construction. Asset growth is expressed as a proportion (e.g., 0.2 = 20% annual growth in total assets) and covers the full sample period of the merged CRSP-Compustat dataset.

With the sorting variable constructed and merged onto the monthly stock panel, we now turn to portfolio sorts, the mechanism that translates firm-level characteristics into tradable factor returns.

## 4.3. Portfolio sorts

In portfolio sorts, stocks are ranked each period by their sorting variable and divided into quantile portfolios. Portfolio returns are equal-weighted or value-weighted averages of constituent stock returns. The long-short factor return is the difference between the extreme portfolios. For asset growth, the relevant spread is bottom minus top, reflecting the negative premium documented by Cooper *et al.* (2008). The construction involves several methodological choices: which stocks enter the universe, how many portfolios to form, which exchange determines the breakpoints, the weighting scheme, and the rebalancing frequency.

`implement_portfolio_sort()` exposes all of these choices through a single function call. The function takes the panel data, one or two sorting variables, a sorting method, and a `portfolio_sort_options` object that bundles two configuration layers: `filter_options()` controls which observations enter the breakpoint and return computation (for example, excluding financial firms or stocks below a minimum price), and `breakpoint_options()` controls how portfolios are formed (number of portfolios, exchange universe, bunching). Internally, the function applies the sample filters and then calls `compute_portfolio_returns()`, which produces the time series of portfolio returns. A subsequent call to `compute_long_short_returns()`

constructs the hedge portfolio.

`compute_portfolio_returns()` itself rests on two lower-level helpers, `compute_breakpoints()` for quantile cut-points and `assign_portfolio()` for mapping each stock-month to a portfolio number. Both are exported, so researchers who need finer control can call them directly rather than through the wrapper. The two examples below use `implement_portfolio_sort()` throughout.

*Baseline specification (univariate sorts)*

The simplest call to `implement_portfolio_sort()` requires only a `portfolio_sort_options` object that specifies how to form the breakpoints. In a univariate sort, stocks are ranked on a single characteristic. We illustrate this here using asset growth as the sorting variable and forming quintile portfolios with breakpoints computed from NYSE-listed stocks only, and rebalanced annually in July.

```
options_baseline <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  )
)
```

The `breakpoint_options()` call specifies that breakpoints come from NYSE-listed stocks only, a common convention that avoids the influence of small AMEX and NASDAQ firms on portfolio cutoffs. We then pass these options to `implement_portfolio_sort()`.

```
portfolio_returns <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = "asset_growth",
  sorting_method = "univariate",
  rebalancing_month = 7,
  portfolio_sort_options = options_baseline
)
```

Apart from setting `rebalancing_month = 7`, every other choice point falls back to the package default: no smooth bunching, no minimum portfolio size, and a cap-weight cutoff at the 80th percentile of market capitalisation for the capped value-weighted return. The standard CRSP/Compustat column names (`permno`, `date`, `ret_excess`, `mktcap_lag`, `exchange`) are picked up automatically through the default `data_options()`. Setting `rebalancing_month = 7` reforms the portfolios each July and holds them for the following twelve months, the standard Fama-French rebalancing convention. This pairs with `ff_adjustment = TRUE` in `join_lagged_values()`: the signal is locked in at the June/July boundary, so July is the first month in which a fresh fiscal-year update is available.

`compute_portfolio_returns()`, called internally, returns a long tibble with one row per portfolio-month and the three weighting columns introduced in Section 3.5 (`ret_excess_vw`, `ret_excess_ew`, `ret_excess_vw_capped`). Piping the result through `compute_long_short_returns()` gives the hedge portfolio. The `direction = "bottom_minus_top"`

argument means the factor is long low-AG stocks and short high-AG stocks, in line with the negative premium documented by Cooper *et al.* (2008).

```
factor_returns_ag <- portfolio_returns |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 4 summarises the time series of each quintile across all three weighting schemes, with the long-short portfolio appended as the final row.

| Portfolio | VW Mean | VW $t$ | EW Mean | EW $t$ | VWC Mean | VWC $t$ |
|---|---|---|---|---|---|---|
| 1 | 0.091 | 4.281 | 0.147 | 4.770 | 0.102 | 4.028 |
| 2 | 0.078 | 4.265 | 0.116 | 5.077 | 0.094 | 4.461 |
| 3 | 0.069 | 3.718 | 0.107 | 4.781 | 0.090 | 4.355 |
| 4 | 0.072 | 3.605 | 0.101 | 4.238 | 0.086 | 3.870 |
| 5 | 0.066 | 2.610 | 0.048 | 1.583 | 0.054 | 1.930 |
| L/S | 0.026 | 1.924 | 0.099 | 9.116 | 0.047 | 4.468 |

Table 4: Time-series averages of annualised monthly excess returns for asset growth quintile portfolios. Portfolios are formed annually in July using NYSE breakpoints. Means are annualised (multiplied by 12). The $t$~statistic tests whether the mean monthly excess return differs from zero. The L/S row is the long-short portfolio (bottom minus top quintile).

The value-weighted long-short portfolio earns 2.6% per year ($t$-statistic 1.92), consistent with the negative premium documented by Cooper *et al.* (2008). The equal-weighted spread is larger, as expected for characteristics that correlate with size. The monotone decline of value-weighted means across quintiles confirms that `tidyfinance` reproduces the literature result from raw WRDS data with a single function call.

*Customised specification (bivariate sorts)*

The baseline specification above takes (almost) every package default and runs a univariate sort to form long-short portfolios. The customised specification below changes both: it overrides several of those defaults through `portfolio_sort_options` and moves from sorting on one variable to sorting on two.

We adopt a specification that combines, in a single call to `implement_portfolio_sort()`, the universe restrictions and breakpoint conventions used in many empirical studies: exclude financials and utilities, exclude firms with non-positive earnings, ignore the smallest 20% of NYSE-sized firms when computing breakpoints, form quintiles on AG within size buckets, use NYSE-only breakpoints, rebalance annually in July, and require at least ten constituents per portfolio.

Bivariate (double) sorts control for a second variable, typically firm size (`mktcap_lag`), when forming portfolios on the main characteristic. We use the dependent variant introduced in Section 3.5 and illustrated in Figure 3: stocks are first split into size buckets, then sorted into asset growth portfolios within each bucket, with returns averaged across the size dimension.

```
options_customised <- portfolio_sort_options(
```

```
  filter_options = filter_options(
    exclude_financials = TRUE,
    exclude_utilities = TRUE,
    exclude_negative_earnings = TRUE
  ),
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE",
    breakpoints_min_size_threshold = 0.2
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
    breakpoints_exchanges = "NYSE"
  )
)

portfolio_returns_customised <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_customised,
  min_portfolio_size = 10,
  quiet = TRUE
)

factor_returns_customised <- portfolio_returns_customised |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

A single object-construction call (`options_customised`) bundles every methodological choice that distinguishes this specification from the baseline; nothing in the surrounding pipeline changes. Switching from a univariate to a size-controlled dependent double sort is a one-word change in `sorting_method`, and the secondary breakpoint object is supplied alongside the primary one without any additional work. The `quiet = TRUE` flag suppresses the messages that `filter_sorting_data()` would otherwise emit reporting how many observations each filter drops; setting `quiet = FALSE` (the default) is useful when iterating to track the sample-construction effects.

For comparison, we also include a bivariate-dependent sort that uses the default filters and breakpoint conventions, so the only difference from the baseline is the addition of size control. The long-short returns from that specification are appended to the customised one in Table 5.

```
options_bivariate <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
```

```
    breakpoints_exchanges = "NYSE"
  )
)

portfolio_returns_bivariate <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_bivariate
)

factor_returns_bivariate <- portfolio_returns_bivariate |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5 decomposes the customised premium into two layers. Adding the size-controlled double sort alone (the bivariate-only row) lifts the annualised premium from 2.6% to 3.6%, with the $t$-statistic moving from borderline to clearly significant. Hence, most of the AG signal in the cross-section was masked by size dispersion in the univariate sort. Layering the filter set and the 20% breakpoint threshold on top trims the magnitude back to 3%, where the spread remains significant at conventional levels: removing financials, utilities, and loss-making firms strips out some of the firms that were contributing to the raw spread. Across all three specifications, the spread is positive and reflects the negative AG premium of Cooper *et al.* (2008).

| Specification | VW Mean | VW $t$ | VW $t$-stat |
|---|---|---|---|
| Baseline (univariate, no filters) | 0.026 | 1.924 | NA |
| Bivariate only (size-controlled, no filters) | 0.036 | NA | 3.376 |
| Customised (size-controlled, filtered) | 0.030 | NA | 2.439 |

Table 5: Three-step decomposition of long-short asset growth returns. The baseline is a univariate sort with NYSE breakpoints and no filters. The bivariate-only specification adds a dependent double sort on size to the baseline but otherwise keeps the same universe and breakpoints. The customised specification adds the filter set (financials, utilities, non-positive earnings) and a 20% size threshold on the breakpoints. All rows use annual July rebalancing; means are annualised value-weighted excess returns.

For researchers who want even finer control, the package exposes the lower-level building blocks (`filter_sorting_data()`, `compute_breakpoints()`, `assign_portfolio()`, `compute_portfolio_returns()`) as standalone functions. They share signatures and semantics with `implement_portfolio_sort()`, so an analysis that prototypes against the wrapper can be unrolled into its components without rewriting the pipeline. This is convenient when custom logic, for example, computing breakpoints separately within each industry, or from a trailing window of past data rather than the contemporaneous cross-section, needs to be applied at one of the stages only.

*Asset pricing tests*

A long-short return is one summary of whether asset growth predicts the cross-section of returns. A complementary view comes from Fama-MacBeth regressions (Fama and MacBeth 1973), which estimate the average risk premium attached to a characteristic after running cross-sectional regressions period by period. `tidyfinance` implements the standard procedure in `estimate_fama_macbeth()`, which takes a panel and a model formula, fits one cross-sectional OLS regression per period, and aggregates the period-level coefficients into average risk premia with Newey-West standard errors by default.

Here, we winsorise the AG signal at the 1st and 99th percentiles to keep extreme observations from dominating the cross-sectional regressions and pair it with log market capitalisation as a size control.

```
fm_data <- sorting_data |>
  filter_out(is.na(asset_growth) | is.na(mktcap_lag) | is.na(ret_excess)) |>
  group_by(date) |>
  mutate(
    asset_growth = winsorize(asset_growth, 0.01),
    log_mktcap = log(mktcap_lag)
  ) |>
  ungroup()

fm_results <- estimate_fama_macbeth(
  fm_data,
  model = "ret_excess ~ asset_growth + log_mktcap"
)
```

| factor | risk_premium | n | standard_error | t_statistic |
|---|---|---|---|---|
| intercept | 0.0158 | 749 | 0.0042 | 3.7701 |
| asset_growth | -0.0065 | 749 | 0.0009 | -7.3908 |
| log_mktcap | -0.0015 | 749 | 0.0004 | -3.4304 |

Table 6: Fama-MacBeth regression of monthly excess returns on lagged asset growth and lagged log market capitalisation. Coefficients are time-series averages of monthly cross-sectional OLS estimates; standard errors are Newey-West with the default lag selection. The intercept and slopes on the right-hand-side variables are reported on the monthly scale.

The negative slope on `asset_growth` lines up with the long-short evidence in Table 4 and Table 5: high-AG firms earn lower subsequent returns, conditional on size. Switching to `vcov = "iid"` or supplying `vcov_options = list(lag = 6, prewhite = FALSE)` would change the standard-error treatment without altering the rest of the call, and adding additional regressors (a beta estimate from `estimate_betas()`, a profitability measure, an idiosyncratic-volatility control) is a one-line extension to the model formula. The same function therefore covers the standard cross-sectional asset-pricing test that complements the portfolio-sort evidence used throughout the rest of the paper.

# 5. Factor library and API

Note

**Section thesis.** Running the package across all defensible combinations of construction choices produces a comprehensive factor library. The library is a side output: it demonstrates that the infrastructure scales and provides a public good for users who want precomputed series.
Argument flow:

1. *Specification grid (`tbl-sorting-decisions`).* The preprocessing decisions varied across the library and their defaults. Cite WWW and **?** on the case for comprehensive grids over a single canonical series.
2. *Coverage.* 50 sorting variables across 841536 specifications (`tbl-sorting-variables`).
3. *Access.* API call (`download_data("tidyfinance", "factor_library", ...)`) and dashboard at factors.tidy-finance.org. Researchers retrieve series without rebuilding the pipeline.
4. *Illustration (`fig-sharpe-ratio`).* Distribution of annualised Sharpe ratios for each variable across specifications. For several variables the premium changes sign across specifications. This is a consequence of the infrastructure, not the paper's contribution.

Mention **py-tidyfinance** once: identical API for Python users.

## 5.1. Specification grid

When every construction choice is configurable, no single combination is canonical. Rather than ship one default series and force users to rebuild the pipeline for every variant, we run **tidyfinance** across all defensible combinations and publish the results as a precomputed factor library. The case for systematic enumeration over construction choices, in preference to any single canonical specification, is made by Walter, Weber, and Weiss (2024) and **?**.

Table 7 lists the preprocessing choices that vary across the library. Italic values mark the default for each choice; non-italic values are the alternatives we run.

| Preprocessing choice | Argument | Options | Description |
|---|---|---|---|
| Size exclusion | `exclude_size` | None; *bottom 20%* | Fraction of the smallest stocks (by NYSE market cap) excluded from the universe. |
| Exclusion of financials | `exclude_financials` | *Yes*; No | Drops financial-sector stocks (SIC 6000–6999) from the universe. |
| Exclusion of utilities | `exclude_utilities` | *No*; Yes | Drops utility-sector stocks (SIC 4900–4999) from the universe. |
| Exclusion of neg. earnings | `exclude_negative_earnings` | *No*; Yes | Drops firms with negative earnings before sorting. |

| Preprocessing choice | Argument | Options | Description |
|---|---|---|---|
| Sorting variable lag | `sorting_variable_lag` | 3; 6; *12*; FF | Lag applied to the sorting variable before portfolio assignment. |
| Rebalancing | `rebalancing` | *Monthly*; Annual | How frequently portfolios are reformed. |
| BP: Quantiles (main) | `breakpoints_main` | 3; 5; *10* | Number of quantile groups for the portfolio sort. |
| Sort method | `sorting_method` | *Single*; Double dependent; Double independent | Single sort or sequential double sort (by size). |
| BP: Quantiles (secondary) | `breakpoints_secondary` | *Median*; 2; 5 | Groups for the secondary sort variable; only relevant for double sorts. |
| BP: Exchanges | `breakpoints_exchanges` | *NYSE*; All | Exchange(s) used to compute quantile breakpoints. |
| Weighting scheme | `weighting_scheme` | EW; *VW*; truncated VW | Return weighting within portfolios. |

Table 7: Preprocessing choices in the portfolio sort grid and corresponding `download_data()` arguments. Italic values indicate the default. For double sorts, the secondary quantiles and exchange choices apply to the size dimension only; for univariate sorts, those rows are not applicable. FF refers to the lag procedure of Fama and French (1993).

The factor library covers 50 sorting variables across seven categories: investment, profitability, valuation, momentum, intangibles, trading frictions, and financing. The selection follows Jensen *et al.* (2023). Table 8 lists all variables with their long-short direction and primary reference. Each variable runs through every valid combination of the choices in Table 7, producing 841536 factor return series in total.

| Variable | Name | | Main reference |
|---|---|---|---|
| 52w | 52-Week High | ↑ | George and Hwang (2004) |
| aci | Abnormal Corporate Investment | ↓ | Titman, Wei, and Xie (2004) |
| ami | Amihud Illiquidity | ↑ | Amihud (2002) |
| ag | Asset Growth | ↓ | Cooper *et al.* (2008) |
| beta | Beta | ↑ | Fama and MacBeth (1973) |
| bm | Book-to-Market | ↑ | Daniel and Titman (2006) |
| cfm | Cash Flow to Market | ↑ | Lakonishok, Shleifer, and Vishny (1994) |
| cbop | Cash-Based Operating Profitability | ↑ | Ball, Gerakos, Linnainmaa, and Nikolaev (2016) |
| dbe | Change in Common Equity | ↓ | Richardson, Sloan, Soliman, and Tuna (2005) |
| dcol | Change in Current Operating Liabilities | ↓ | Richardson *et al.* (2005) |

| | | | |
|---|---|---|---|
| dfnl | Change in Financial Liabilities | ↓ | Richardson *et al.* (2005) |
| dnoa | Change in Net Operating Assets | ↓ | Hirshleifer, Hou, Teoh, and Zhang (2004) |
| dpia | Change in PPE and Intangibles | ↓ | Lyandres, Sun, and Zhang (2008) |
| dwc | Change in Working Capital | ↓ | Richardson *et al.* (2005) |
| cdi | Composite Debt Issuance | ↓ | Lyandres *et al.* (2008) |
| csi | Composite Share Issuance | ↓ | Daniel and Titman (2006) |
| dm | Debt to Market | ↑ | Bhandari (1988) |
| dtv | Dollar Trading Volume | ↓ | Brennan, Chordia, and Subrahmanyam (1998) |
| em | Earnings to Market | ↑ | Basu (1983) |
| ebm | Enterprise Book to Market | ↑ | Penman, Richardson, and Tuna (2007) |
| gpa | Gross Profits to Assets | ↑ | Novy-Marx (2013) |
| hr | Hiring Rate | ↓ | Belo, Lin, and Bazdresch (2014) |
| iskew | Idiosyncratic Skewness | ↓ | Bali *et al.* (2016) |
| ivol | Idiosyncratic Volatility | ↓ | Ang, Hodrick, Xing, and Zhang (2006) |
| dinv | Inventory Changes | ↓ | Thomas and Zhang (2002) |
| ig | Investment Growth | ↓ | Xing (2008) |
| rev | Long-Term Reversal | ↓ | De Bondt and Thaler (1985) |
| me | Market Equity | ↓ | Banz (1981) |
| mdr | Maximum Daily Return | ↓ | Bali, Cakici, and Whitelaw (2011) |
| mom | Momentum | ↑ | Fama and French (1996) |
| ndf | Net Debt Financing | ↓ | Baker and Wurgler (2006) |
| ndm | Net Debt to Market | ↓ | Penman *et al.* (2007) |
| nef | Net Equity Financing | ↓ | Baker and Wurgler (2006) |
| nxf | Net External Financing | ↓ | Baker and Wurgler (2006) |
| noa | Net Operating Assets | ↓ | Hirshleifer *et al.* (2004) |
| npy | Net Payout Yield | ↑ | Boudoukh, Michaely, Richardson, and Roberts (2007) |
| o | O-Score | ↓ | Ohlson (1980); Dichev (1998) |
| oa | Operating Accruals | ↓ | Sloan (1996) |
| ocm | Operating Cash Flow to Market | ↑ | Desai, Rajgopal, and Venkatachalam (2004) |
| ope | Operating Efficiency | ↑ | Fama and French (2015) |
| ol | Operating Leverage | ↑ | Novy-Marx (2011) |
| rer | Real-Estate Ratio | ↑ | Titman *et al.* (2004) |
| rs | Relative Strength | ↑ | Jegadeesh and Livnat (2006) |
| rmom | Residual Momentum | ↑ | Blitz, Huij, and Martens (2011) |
| roa | Return on Assets | ↑ | Balakrishnan, Bartov, and Faurel (2010) |
| roe | Return on Equity | ↑ | Hou, Xue, and Zhang (2014) |
| sm | Sales to Market | ↑ | Barbee Jr, Mukherji, and Raines (1996) |
| tur | Share Turnover | ↓ | Datar, Naik, and Radcliffe (1998) |
| sue | Standardized Unexpected Earnings | ↑ | Foster, Olsen, and Shevlin (1984) |

| | | | |
|---|---|---|---|
| tbi | Taxable Income to Book Income | ↓ | Lev and Nissim (2004) |

Table 8: Sorting variables in the factor library. The direction column indicates whether the long-short factor return goes long top-minus-bottom (↑) or bottom-minus-top (↓) of the characteristic distribution.

### 5.2. Access

Researchers retrieve precomputed series without rebuilding the pipeline. **tidyfinance** hosts the library on Hugging Face and exposes it through `download_data()`. An interactive dashboard at factors.tidy-finance.org provides the same series through a browser interface. The companion Python package **py-tidyfinance** offers an identical API for users who prefer Python.

A default call retrieves a single factor by name and lag:

```
asset_growth_returns <- download_data(
  "tidyfinance",
  "factor_library",
  sorting_variable = "ag",
  sorting_variable_lag = "6m"
)
```

The defaults passed to `dataset = "factor_library"` reflect one common construction choice but may not suit every research question. Verify that the selected combination matches the intended design before drawing conclusions.

```
download_data(
  "tidyfinance",
  "factor_library",
  sorting_variable = "52w",
  sorting_variable_lag = "6m",
  fill_all = TRUE
)
```

For finer control, **tidyfinance** exposes lower-level helpers that sit beneath the `download_data()` interface. `get_available_huggingface_files()` returns all available files for a given Hugging Face repository as a tibble; `download_factor_library_returns_ids()` retrieves returns for a specific vector of precomputed specification IDs. Researchers who want to iterate over a custom subset of specifications rather than filter by parameter value will prefer this route.

```
full_grid <- get_available_huggingface_files(
  "tidy-finance",
  "factor-library-grid"
) |>
  dplyr::pull(.data$url) |>
  arrow::read_parquet()

tidyfinance:::download_factor_library_ids(123)
```

### 5.3. Construction-choice sensitivity

The library makes systematic sensitivity analysis tractable. As a demonstration of what the infrastructure produces at scale, Figure 5 shows the distribution of annualised Sharpe ratios for each sorting variable across all 841536 specifications.

Two patterns are visible. Most distributions are wide, indicating that construction choices have material effects on factor performance across the library. For several variables the distribution spans zero, meaning the sign of the premium depends on the specification. These patterns are a consequence of the infrastructure operating at scale, not a finding of this paper. The shape of each distribution is itself an output that researchers can interrogate when designing their own sensitivity analyses.

## 6. Summary

Note

**Section thesis (three beats, target ~150 words total):**

1. *What the package delivers.* **tidyfinance** makes every construction decision in empirical asset pricing visible, configurable, and reproducible from raw data to factor return.
2. *Commitment.* The package will be maintained and extended. Concrete next steps: international equities, additional factor methodologies, expanded fixed-income coverage.
3. *Broader mission.* Reproducible quantitative finance and more effective teaching of empirical methods, in line with tidy-finance.org pedagogical materials.

Do not restate the abstract. Do not introduce new findings. Do not embed a grant application.
The existing prose below this note does not yet match these three beats and will need a rewrite.

Empirical asset pricing depends on dozens of construction choices that no consensus pins down. **tidyfinance** makes those choices first-class objects in the workflow: each is an argument, each has a documented default, each can be varied without rebuilding the pipeline. Disagreements over construction become empirical questions, not implicit assumptions buried in code.

We will maintain and extend the package on CRAN. The current release covers US equities through CRSP and Compustat; the next milestones add international equity panels, additional factor methodologies, and expanded fixed-income coverage beyond TRACE.

Beyond this paper, **tidyfinance** is part of a broader effort toward reproducible quantitative finance and more effective teaching of empirical methods. Companion materials at tidy-finance.org integrate the package into open coursework and self-study tracks for graduate finance and asset pricing.

## Computational details

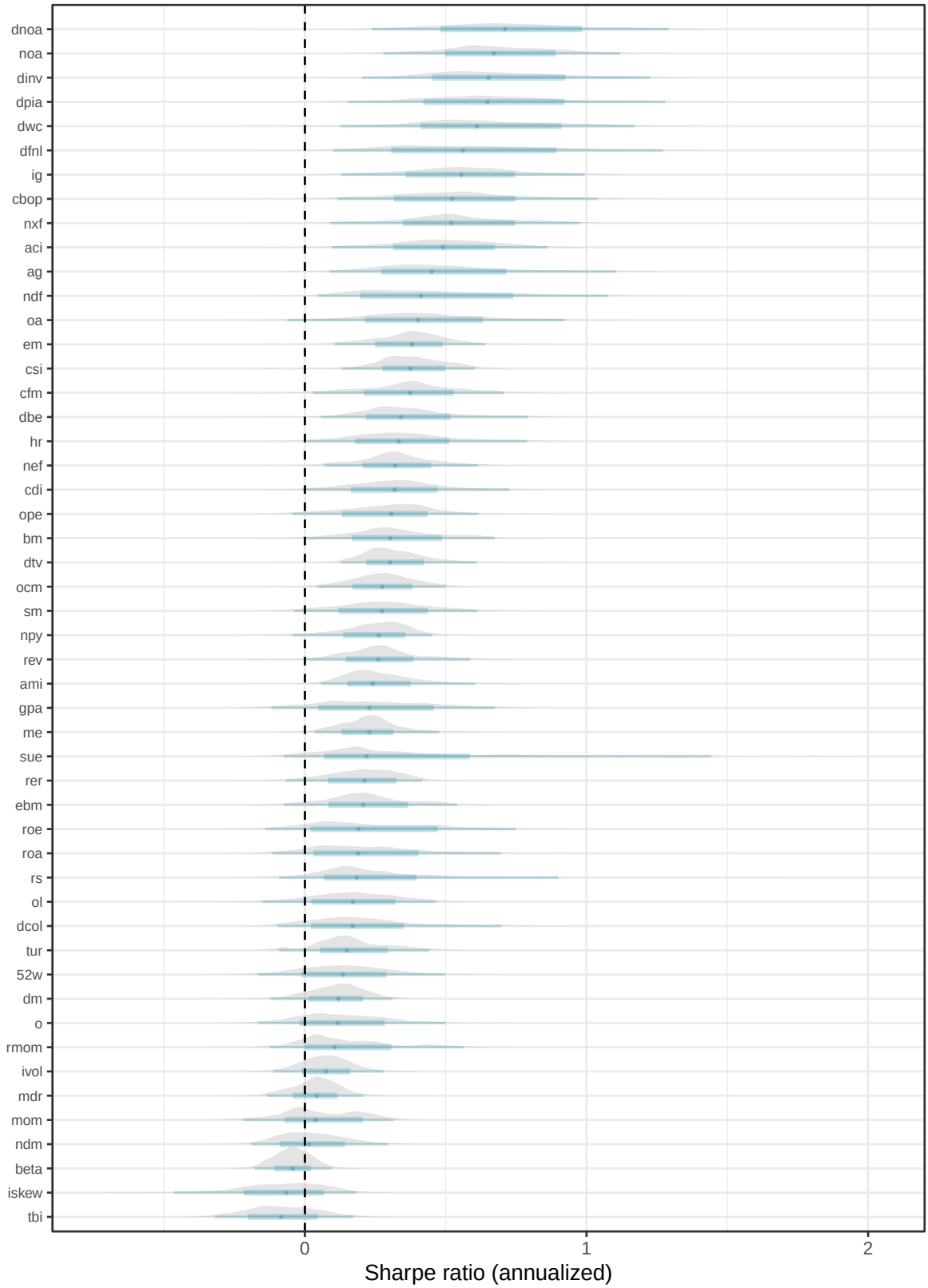



Figure 5: Sharpe ratios across construction choices. Ridge plots show the distribution of annualised Sharpe ratios of long-short return differentials for each sorting variable across all `r .QuartoInlineRender(nrow(portfolio_sort_grid))` specifications illustrated in Table 7, based on the available sample period per factor.

We obtained the results in this paper using R~4.5.2 with the following packages: **tidyfinance**~0.4.5.9027, **dplyr**~1.2.0, **tidyr**~1.3.1, **ggplot2**~4.0.0, **lubridate**~1.9.4, **arrow**~21.0.0.1, **stringr**~1.5.2, **forcats**~1.0.0, **ggdist**~3.3.3, and **knitr**~1.51. CRAN hosts R and all packages at https://CRAN.R-project.org/. **renv**~1.1.5 ensures full reproducibility of the computational environment. Reproducing the factor return data requires an institutional subscription to Wharton Research Data Services (WRDS), as the underlying CRSP and Compustat data are proprietary and cannot be redistributed.

Reproducing this paper from cached intermediate data takes about 15 minutes on a regular workstation. Recomputing the full factor library from raw WRDS data across all 841536 specifications takes about 12 hours on a 64-core machine.

We publish all code, data workflows, and documentation at https://github.com/tidy-finance/jss-multilingual-factor-library. The code is released under the MIT License, which is compatible with the GNU General Public License (GPL).

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: Europe/Copenhagen
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices datasets  utils     methods   base

other attached packages:
[1] patchwork_1.3.2        ggforce_0.5.0          ggplot2_4.0.0
[4] lubridate_1.9.4        tidyr_1.3.1            dplyr_1.2.0
[7] tidyfinance_0.4.5.9027

loaded via a namespace (and not attached):
 [1] ggdist_3.3.3         sandwich_3.1-1       rappdirs_0.3.3
 [4] generics_0.1.4       renv_1.1.5           stringi_1.8.7
 [7] lattice_0.22-7       hms_1.1.4            digest_0.6.37
[10] magrittr_2.0.4       RColorBrewer_1.1-3   evaluate_1.0.5
[13] grid_4.5.2           timechange_0.3.0     fastmap_1.2.0
```

```
[16] blob_1.3.0           jsonlite_2.0.0       RPostgres_1.4.9
[19] DBI_1.2.3            purrr_1.1.0          scales_1.4.0
[22] frenchdata_0.2.0     tweenr_2.0.3         httr2_1.2.2
[25] cli_3.6.5            rlang_1.1.7          polyclip_1.10-7
[28] dbplyr_2.5.2.9000    bit64_4.6.0-1        withr_3.0.2
[31] yaml_2.3.10          tools_4.5.2          tzdb_0.5.0
[34] curl_7.0.0           assertthat_0.2.1     vctrs_0.7.2
[37] R6_2.6.1             zoo_1.8-15           lifecycle_1.0.5
[40] stringr_1.5.2        bit_4.6.0            MASS_7.3-65
[43] arrow_21.0.0.1       pkgconfig_2.0.3      pillar_1.11.1
[46] gtable_0.3.6         Rcpp_1.1.0           glue_1.8.0
[49] xfun_0.53            tibble_3.3.0         tidyselect_1.2.1
[52] knitr_1.51           farver_2.1.2         htmltools_0.5.8.1
[55] labeling_0.4.3       rmarkdown_2.29       compiler_4.5.2
[58] S7_0.2.0             distributional_0.7.0
```

## Acknowledgments

Stefan Voigt is grateful to the Danish Finance Institute for financial support.

## References

Amihud Y (2002). "Illiquidity and Stock Returns: Cross-Section and Time-Series Effects." *Journal of Financial Markets*, **5**(1), 31–56.

Ang A, Hodrick RJ, Xing Y, Zhang X (2006). "The Cross-Section of Volatility and Expected Returns." *The Journal of Finance*, **61**(1), 259–299.

Baker M, Wurgler J (2006). "Investor Sentiment and the Cross-Section of Stock Returns." *The Journal of Finance*, **61**(4), 1645–1680.

Balakrishnan K, Bartov E, Faurel L (2010). "Post Loss/Profit Announcement Drift." *Journal of Accounting and Economics*, **50**(1), 20–41.

Bali TG, Cakici N, Whitelaw RF (2011). "Maxing Out: Stocks as Lotteries and the Cross-Section of Expected Returns." *Journal of Financial Economics*, **99**(2), 427–446.

Bali TG, Engle RF, Murray S (2016). *Empirical Asset Pricing: The Cross Section of Stock Returns*. John Wiley & Sons.

Ball R, Gerakos J, Linnainmaa JT, Nikolaev V (2016). "Accruals, Cash Flows, and Operating Profitability in the Cross Section of Stock Returns." *Journal of Financial Economics*, **121**(1), 28–45.

Banz RW (1981). "The Relationship Between Return and Market Value of Common Stocks." *Journal of Financial Economics*, **9**(1), 3–18.

Barbee Jr WC, Mukherji S, Raines GA (1996). "Do Sales–Price and Debt–Equity Explain Stock Returns Better Than Book–Market and Firm Size?" *Financial Analysts Journal*, **52**(2), 56–60.

Basu S (1983). "The Relationship Between Earnings' Yield, Market Value and Return for NYSE Common Stocks: Further Evidence." *Journal of Financial Economics*, **12**(1), 129–156.

Belo F, Lin X, Bazdresch S (2014). "Labor Hiring, Investment, and Stock Return Predictability in the Cross Section." *Journal of Political Economy*, **122**(1), 129–177.

Bhandari LC (1988). "Debt/Equity Ratio and Expected Common Stock Returns: Empirical Evidence." *The Journal of Finance*, **43**(2), 507–528.

Blitz D, Huij J, Martens M (2011). "Residual Momentum." *Journal of Empirical Finance*, **18**(3), 506–521.

Boudoukh J, Michaely R, Richardson M, Roberts MR (2007). "On the Importance of Measuring Payout Yield: Implications for Empirical Asset Pricing." *The Journal of Finance*, **62**(2), 877–915.

Bradshaw MT, Richardson SA, Sloan RG (2006). "The Relation Between Corporate Financing Activities, Analysts' Forecasts and Stock Returns." *Journal of Accounting and Economics*, **42**(1-2), 53–85.

Brennan MJ, Chordia T, Subrahmanyam A (1998). "Alternative Factor Specifications, Security Characteristics, and the Cross-Section of Expected Stock Returns." *Journal of Financial Economics*, **49**(3), 345–373.

Chen A, Zimmermann T (2022). "Open Source Cross-Sectional Asset Pricing." *Critical Finance Review*, **27**(2), 207–264.

Cooper MJ, Gulen H, Schill MJ (2008). "Asset Growth and the Cross-Section of Stock Returns." *The Journal of Finance*, **63**(4), 1609–1651.

Dancho M, Vaughan D (2026). *tidyquant: Tidy Quantitative Financial Analysis*. doi:10.32614/CRAN.package.tidyquant. R package version 1.0.12, URL https://CRAN.R-project.org/package=tidyquant.

Daniel K, Titman S (2006). "Market Reactions to Tangible and Intangible Information." *The Journal of Finance*, **61**(4), 1605–1643.

Datar VT, Naik NY, Radcliffe R (1998). "Liquidity and Stock Returns: An Alternative Test." *Journal of Financial Markets*, **1**(2), 203–219.

Davis JL, Fama EF, French KR (2000). "Characteristics, Covariances, and Average Returns: 1929 to 1997." *The Journal of Finance*, **55**(1), 389–406.

De Bondt WF, Thaler R (1985). "Does the Stock Market Overreact?" *The Journal of Finance*, **40**(3), 793–805.

Desai H, Rajgopal S, Venkatachalam M (2004). "Value-Glamour and Accruals Mispricing: One Anomaly or Two?" *The Accounting Review*, **79**(2), 355–385.

Dichev ID (1998). "Is the Risk of Bankruptcy a Systematic Risk?" *The Journal of Finance*, **53**(3), 1131–1147.

Fama EF, French KR (1992). "The Cross-Section of Expected Stock Returns." *The Journal of Finance*, **47**(2), 427–465.

Fama EF, French KR (1993). "Common Risk Factors in the Returns on Stocks and Bonds." *Journal of Financial Economics*, **33**(1), 3–56.

Fama EF, French KR (1996). "Multifactor Explanations of Asset Pricing Anomalies." *The Journal of Finance*, **51**(1), 55–84.

Fama EF, French KR (2015). "A Five-Factor Asset Pricing Model." *Journal of Financial Economics*, **116**(1), 1–22.

Fama EF, MacBeth JD (1973). "Risk, Return, and Equilibrium: Empirical Tests." *Journal of Political Economy*, **81**(3), 607–636.

Foster G, Olsen C, Shevlin T (1984). "Earnings Releases, Anomalies, and the Behavior of Security Returns." *The Accounting Review*, **59**(4), 574–603.

Gao X, Ritter JR (2010). "The Marketing of Seasoned Equity Offerings." *Journal of Financial Economics*, **97**(1), 33–52.

George TJ, Hwang CY (2004). "The 52-Week High and Momentum Investing." *The Journal of Finance*, **59**(5), 2145–2176.

Hirshleifer D, Hou K, Teoh SH, Zhang Y (2004). "Do Investors Overvalue Firms with Bloated Balance Sheets?" *Journal of Accounting and Economics*, **38**, 297–331.

Hou K, Xue C, Zhang L (2014). "Digesting Anomalies: An Investment Approach." *The Review of Financial Studies*, **28**(3), 650–705.

Hou K, Xue C, Zhang L (2020). "Replicating Anomalies." *The Review of Financial Studies*, **33**(5), 2019–2133.

Hribar P, Collins DW (2002). "Errors in Estimating Accruals: Implications for Empirical Research." *Journal of Accounting Research*, **40**(1), 105–134.

Jegadeesh N, Livnat J (2006). "Revenue Surprises and Stock Returns." *Journal of Accounting and Economics*, **41**(1-2), 147–171.

Jensen TI, Kelly B, Pedersen LH (2023). "Is There a Replication Crisis in Finance?" *The Journal of Finance*, **78**(5), 2465–2518.

Lakonishok J, Shleifer A, Vishny RW (1994). "Contrarian Investment, Extrapolation, and Risk." *The Journal of Finance*, **49**(5), 1541–1578.

Lev B, Nissim D (2004). "Taxable Income, Future Earnings, and Equity Values." *The Accounting Review*, **79**(4), 1039–1074.

Lyandres E, Sun L, Zhang L (2008). "The New Issues Puzzle: Testing the Investment-Based Explanation." *The Review of Financial Studies*, **21**(6), 2825–2855.

Novy-Marx R (2011). "Operating Leverage." *Review of Finance*, **15**(1), 103–134.

Novy-Marx R (2013). "The Other Side of Value: The Gross Profitability Premium." *Journal of Financial Economics*, **108**(1), 1–28.

Ohlson JA (1980). "Financial Ratios and the Probabilistic Prediction of Bankruptcy." *Journal of Accounting Research*, **18**(1), 109–131.

Penman SH, Richardson SA, Tuna I (2007). "The Book-to-Price Effect in Stock Returns: Accounting for Leverage." *Journal of Accounting Research*, **45**(2), 427–467.

Peterson BG, Carl P (2026). *PerformanceAnalytics: Econometric Tools for Performance and Risk Analysis*. doi:10.32614/CRAN.package.PerformanceAnalytics. R package version 2.1.0, URL https://CRAN.R-project.org/package=PerformanceAnalytics.

Richardson SA, Sloan RG, Soliman MT, Tuna I (2005). "Accrual Reliability, Earnings Persistence and Stock Prices." *Journal of Accounting and Economics*, **39**(3), 437–485.

Ryan JA, Ulrich JM (2025). *quantmod: Quantitative Financial Modelling Framework*. doi:10.32614/CRAN.package.quantmod. R package version 0.4.28, URL https://CRAN.R-project.org/package=quantmod.

Sloan RG (1996). "Do Stock Prices Fully Reflect Information in Accruals and Cash Flows about Future Earnings?" *The Accounting Review*, **71**(3), 289–315.

Thomas JK, Zhang H (2002). "Inventory Changes and Future Returns." *Review of Accounting Studies*, **7**(2), 163–187.

Titman S, Wei KJ, Xie F (2004). "Capital Investments and Stock Returns." *Journal of Financial and Quantitative Analysis*, **39**(4), 677–700.

Tuzel S (2010). "Corporate Real Estate Holdings and the Cross-Section of Stock Returns." *The Review of Financial Studies*, **23**(6), 2268–2302.

Walter D, Weber R, Weiss P (2024). "Methodological Uncertainty in Portfolio Sorts." *Available at SSRN 4164117*.

Wickham H, François R, Henry L, Müller K, Vaughan D (2026). *dplyr: A Grammar of Data Manipulation*. doi:10.32614/CRAN.package.dplyr. R package version 1.2.0, URL https://CRAN.R-project.org/package=dplyr.

Xing Y (2008). "Interpreting the Value Effect Through the Q-Theory: An Empirical Investigation." *The Review of Financial Studies*, **21**(4), 1767–1795.

Internet Appendix

# IA-1. Variable construction

In this section, we show the construction details for the risk factors. We group them by their underlying mechanism as shown in Table 8 in the Appendix of the main paper.

## IA-1.1. Financing

### CDI - Composite Debt Issuance

Following Lyandres *et al.* (2008), we compute composite debt issuance as the logarithmic growth rate of book debt (i.e., current debt, DLC, plus long-term debt, DLTT) over the last five years, i.e.,

$$\text{CDI}_t = \log(\text{DLC}_t + \text{DLTT}_t) - \log(\text{DLC}_{t-60} + \text{DLTT}_{t-60})$$

We replace the missing values of DLC and DLTT with zero individually. Additionally, the sum of DLC and DLTT must be larger than zero, and is set to missing otherwise.

### CSI - Composite Share Issuance

Following Daniel and Titman (2006), we measure composite share issuance from CRSP data as the difference between the change in market equity (ME) (i.e., the shares outstanding, shrout, times the absolute value of their price, altprc, in million USD) and the cumulative log return (where $r$ is a simple return) in each month from $t$ to $t-60$, i.e.,

$$\text{CSI}_t = \log(ME_t/ME_{t-60}) - \sum_{\tau=t-59}^{t} \log(1+r_\tau)$$

### DBE - Change in Common Equity

Following Richardson *et al.* (2005), we calculate the change in common equity as the change in common equity (CEQ) over one year scaled by lagged assets (AT), i.e.,

$$DBE_t = \frac{CEQ_t - CEQ_{t-12}}{AT_{t-12}}$$

### DCOL - Change in Current Operating Liabilities

Following Richardson *et al.* (2005), we measure the change in current operating liabilities as the change in current operating liabilities (i.e., current liabilities, LCT, minus current debt, DLC) over one year scaled by lagged assets (AT), i.e.,

$$DCOL_t = \frac{(LCT_t - DLC_t) - (LCT_{t-12} - DLC_{t-12})}{AT_{t-12}}$$

We replace the missing values of DLC with 0.

**DFNL - Change in Financial Liabilities**

Following Richardson *et al.* (2005), we measure the change in financial liabilities (i.e., the sum of current debt, DLC, plus long-term debt, DLTT, plus preferred stocks, PSTK) over one year scaled by lagged assets (AT), i.e.,

$$DFNL_t = \frac{(DLC_t + DLTT_t + PSTK_t) - (DLC_{t-12} + DLTT_{t-12} + PSTK_{t-12})}{AT_{t-12}}$$

We replace the missing values of DLC, DLTT and PSTK with 0, if at least one of them is given.

**NDF - Net Debt Financing**

Following Bradshaw, Richardson, and Sloan (2006), we measure net debt financing as cash proceeds from the issuance of long-term debt (DLTIS) minus cash payments for long-term debt reductions (DLTR) plus net changes in current debt (DLCCH) scaled by the average total assets (AT) over the current and the last year, i.e.,

$$NDF_t = \frac{DLTIS_t - DLTR_t + DLCCH_t}{\frac{1}{2}(AT_t + AT_{t-12})}$$

We replace the missing values of DLCCH with zero.

**NEF - Net Equity Financing**

Following Bradshaw *et al.* (2006), we measure net equity financing as proceeds from the sale of common and preferred stocks (SSTK) minus payments for the repurchase of common and preferred stocks (PRSTKC) minus cash payments for dividends (DV) scaled by the average total assets (AT) over the current and the last year, i.e.,

$$NEF_t = \frac{SSTK_t - PRSTKC_t - DV_t}{\frac{1}{2}(AT_t + AT_{t-12})}$$

**NXF - Net External Financing**

Following Bradshaw *et al.* (2006), we measure net external financing as the sum of net debt financing (NDF) and net equity financing (NEF) as defined above.

## IA-1.2. Intangibles

**HR - Hiring Rate**

Following Belo *et al.* (2014), we measure the hiring rate as the change in the number of employees (EMP) over one year scaled by the average number of employees, i.e.,

$$HR_t = \frac{EMP_t - EMP_{t-12}}{\frac{1}{2}(EMP_t + EMP_{t-12})}$$

We exclude firms with a hiring rate of zero.

**OL - Operating Leverage**

Following Novy-Marx (2011), we measure operating leverage as cost of goods sold (COGS) plus selling, general, and administrative expenses (XSGA), both scaled by current total assets (AT), i.e.,

$$OL_t = \frac{COGS_t + XSGA_t}{AT_t}$$

**RER - Real-Estate Ratio**

Following Tuzel (2010), we measure the real-estate ratio in two subperiods differently.

(a) Prior to 1983, the real-estate ratio is the sum of buildings (PPENB) and capital leases (PPENLS) scaled by net property, plant, and equipment (PPENT), i.e.,

$$RER_t = \frac{PPENB_t + PPENLS_t}{PPENT_t}$$

(b) After the end of 1983, the real-estate ratio is the sum of buildings at cost (FATB) and leases at cost (FATL), both divided by gross property, plant, and equipment (PPEGT), i.e.,

$$RER_t = \frac{FATB_t + FATL_t}{PPEGT_t}$$

Subsequently, we winsorize the real-estate ratios in each fiscal year at the first and 99th percentile. The industry-adjusted real-estate ratio is obtained by subtracting the industry average real-estate ratio from each stock-specific real-estate ratio. We use 2-digit SIC codes to assign stocks to industries. We always require at least five observations to calculate the industry average each year.

## IA-1.3. Investment

**ACI - Abnormal Corporate Investment**

Following Titman *et al.* (2004), we measure abnormal corporate investments using the variable CE, which is capital expenditures (CAPX) scaled by sales (SALE), then ACI is given by CE scaled by CE from the previous three years minus one, i.e.,

$$ACI_t = \frac{CE_t}{\frac{1}{3}(CE_{t-12} + CE_{t-24} + CE_{t-36})} - 1$$

We follow Hou *et al.* (2020) and exclude stocks with sales below 10 million dollars.

**AG - Asset Growth**

Following Cooper *et al.* (2008), we measure asset growth as the change in total assets (AT) over one year scaled by lagged assets (AT), i.e.,

$$AG_t = \frac{AT_t - AT_{t-12}}{AT_{t-12}}$$

**DINV - Inventory Changes**

Following Thomas and Zhang (2002), we measure the change in inventory as the annual change in inventories (INVT) scaled by average total assets (AT) over the current and the last year, i.e.,

$$DINV_t = \frac{INVT_t - INVT_{t-12}}{\frac{1}{2}(AT_t + AT_{t-12})}$$

**DNOA - Change in Net Operating Assets**

Following Hirshleifer *et al.* (2004), we measure net operating assets (defined as $NOA^{int}$) as

$$NOA_t^{int} = (AT_t - CHE_t) - (AT_t - DLC_t - DLTT_t - MIB_t - PSTK_t - CEQ_t)$$

where AT corresponds to total assets, CHE to cash and short-term investments, DLC to current debt, DLTT to long-term debt, MIB to minority interests, PSTK to the value of preferred stocks, and CEQ to common equity. Missing values in DLC, DLTT, MIB, and PSTK are set to zero. Then, the change in net operating assets is the difference between net operating assets of fiscal year $t$ and fiscal year $t-12$ scaled by lagged assets (AT), i.e.,

$$DNOA_t = \frac{NOA_t^{int} - NOA_{t-12}^{int}}{AT_{t-12}}$$

**DPIA - Change in Property, Plant, Equipment, and Inventory**

Following Lyandres *et al.* (2008), we measure the change in property, plant, equipment, and inventory by combining the change in gross property, plant, and equipment (PPEGT) with the annual change in inventory (INVT) and scaling this sum by lagged total assets (AT), i.e.,

$$DPIA_t = \frac{(PPEGT_t - PPEGT_{t-12}) + (INVT_t - INVT_{t-12})}{AT_{t-12}}$$

**DWC - Change in Net Non-Cash Working Capital**

Following Richardson *et al.* (2005), we measure non-cash working capital as

$$WC_t = (ACT_t - CHE_t) - (LCT_t - DLC_t)$$

where ACT corresponds to current assets, CHE to cash, LCT to current liabilities, and DLC to short-term debt. We set missing values of DLC to zero. Then, the change in net non-cash

working capital corresponds to the change in $WC$ from fiscal year $t-12$ to fiscal year $t$ scaled by lagged total assets (AT), i.e.,

$$DWC_t = \frac{WC_t - WC_{t-12}}{AT_{t-12}}$$

**IG - Investment Growth**

Following Xing (2008), we measure investment growth as the annual change in capital expenditures (CAPX) scaled by lagged capital expenditures, i.e.,

$$IG_t = \frac{CAPX_t - CAPX_{t-12}}{CAPX_{t-12}}$$

**NOA - Net Operating Assets**

Following Hirshleifer *et al.* (2004), we measure net operating assets (defined as $NOA^{int}$) as

$$NOA_t^{int} = (AT_t - CHE_t) - (AT_t - DLC_t - DLTT_t - MIB_t - PSTK_t - CEQ_t)$$

where AT corresponds to total assets, CHE to cash and short-term investments, DLC to current debt, DLTT to long-term debt, MIB to minority interests, PSTK to the value of preferred stocks, and CEQ to common equity. Missing values in DLC, DLTT, MIB, and PSTK are set to zero. Then, net operating assets is the net operating assets of fiscal year $t$ scaled by lagged assets (AT), i.e.,

$$NOA_t = \frac{NOA_t^{int}}{AT_{t-12}}$$

**OA - Operating Accruals**

Following Sloan (1996) (before 1988) and Hribar and Collins (2002) (after 1988), we measure operating accruals until 1988 as

$$OA_t = \frac{(\Delta ACT_t - \Delta CHE_t) - (\Delta LCT_t - \Delta DLC_t - \Delta TXP_t) - DP_t}{AT_{t-12}}$$

where ACT is current assets, CHE cash, LCT current liabilities, DLC short-term debt, TXP taxes payable, and DP depreciation and amortization. Moreover, we replace the missing values of DLC and TXP with zero. $\Delta$ represents changes over one year. From 1988 onwards, we measure operating accruals as

$$OA_t = \frac{NI_t - OANCF_t}{AT_{t-12}}$$

where NI is net income and OANCF corresponds to net cash flow from operations.

## IA-1.4. Momentum

### MOM - Return Momentum

Following Fama and French (1996), we measure return momentum from monthly CRSP returns (where $r$ is a simple return) as the cumulative return from month $t-11$ to month $t-1$ skipping the most recent month, i.e.,

$$\text{MOM}_t = \exp\left(\sum_{\tau=t-11}^{t-1} \log(1+r_\tau)\right) - 1$$

### RMOM - Residual Momentum

Following Blitz *et al.* (2011), we define the 11-month residual momentum (RMOM) in each month and for each stock as cumulative residual returns from month $t-11$ to month $t-1$, scaled by the standard deviation of residual returns over the same time horizon. Residual returns $\epsilon$ are obtained in each month from regressing monthly simple, excess stock returns from month $t-1$ to month $t-36$ on the Fama and French (1993) three-factor model. Throughout these rolling regressions, we always require 36 monthly returns.

### RS - Revenue Surprise

Following Jegadeesh and Livnat (2006), we measure the change in revenues per share (DRPS) from quarterly Compustat data as quarterly sales (SALEQ) scaled by the product of quarterly shares outstanding (CSHPRQ) times their cumulative adjustment factor (AJEXQ), i.e.,

$$DRPS_q = \frac{SALEQ_q}{CSHPRQ_q \cdot AJEXQ_q} - \frac{SALEQ_{q-4}}{CSHPRQ_{q-4} \cdot AJEXQ_{q-4}}$$

Then, the revenue surprise corresponds to the change in revenues per share over the last four quarters scaled by the standard deviation of the change in revenues per share over the last eight quarters. We require at least six quarterly observations for this rolling standard deviation and remove infinite values. Finally, the earnings announcement date has to be after the fiscal quarter end date.[3]

### SUE - Standardized Unexpected Earnings

Following Foster *et al.* (1984), we measure unexpected earnings per share (UES) from quarterly Compustat data as the change in split-adjusted earnings (which are equal to quarterly earnings per share, EPSPXQ, scaled by the correction factor for shares outstanding, AJEXQ) per share from its value four quarters ago

$$UES_q = \frac{EPSPXQ_q}{AJEXQ_q} - \frac{EPSPXQ_{q-4}}{AJEXQ_{q-4}}$$

Then, standardized unexpected earnings are defined as unexpected earnings per share divided by the standard deviation of unexpected earnings per share over the previous eight quarters.

[3]We follow the convention of Hou *et al.* (2020) and do not estimate a random walk model with drift to compute the surprise.

We require at least six quarterly observations for this rolling standard deviation. Finally, the earnings announcement date has to be after the fiscal quarter end date.[4]

**52W - 52-Week High**

Following George and Hwang (2004), we measure the 52-week high as month $t$'s closing split-adjusted stock price scaled by the highest daily split-adjusted stock price over the previous twelve months. We only consider stock-month pairs with more than three valid price observations.

**REV - Long-Term Reversal**

Following De Bondt and Thaler (1985), we measure the long-term reversal effect by the cumulative returns from month $t-60$ to month $t-13$, i.e.,

$$\text{REV}_t = \exp\left(\sum_{\tau=t-60}^{t-13} \log(1+r_\tau)\right) - 1$$

## IA-1.5. Profitability

**CBOP - Cash-Based Operating Profitability**

Following Ball *et al.* (2016), we measure cash-based operating profitability

$$\begin{aligned} CBOP_t = &(REVT_t - COGS_t - XSGA_t + XRD_t \\ &- \Delta RECT_t - \Delta INVT_t - \Delta XPP_t + \Delta DRC_t \\ &+ \Delta DRLT_t + \Delta AP_t + \Delta XACC_t)/AT_t \end{aligned}$$

where REVT is total revenue, COGS are cost of goods sold, XSGA are selling, general, and administrative expenses, and XRD are R&D expenses. Moreover, $\Delta$ refers to one-year changes, which we measure for accounts receivable (RECT), inventory (INVT), prepaid expenses (XPP), deferred revenues (the sum of current, DRC, and long-term, DRLT), trade accounts payable (AP), and accrued expenses (XACC). We set the missing values of XRD and all missing changes to zero.

**GPA - Gross Profits to Assets**

Following Novy-Marx (2013), we measure gross profits to assets as total revenues (REVT) minus cost of goods sold (COGS) scaled by current total assets (AT), i.e.,

$$GPA_t = \frac{REVT_t - COGS_t}{AT_t}$$

**O - Ohlson's O-score**

[4]We follow the convention of Hou *et al.* (2020) and do not estimate a random walk model with drift to compute the surprise.

Following Ohlson (1980), we measure O-score with the following linear relation

$$\begin{aligned} O_t = &- 1.32 - 0.407 \cdot \log(AT_t) + 6.03 \cdot \frac{DLC_t + DLTT_t}{AT_t} \\ &- 1.43 \cdot \frac{ACT_t - LCT_t}{AT_t} + 0.076 \cdot \frac{LCT_t}{AT_t} \\ &- 1.72 \cdot \mathbf{1}_{LT_t > AT_t} - 2.37 \cdot \frac{NI_t}{AT_t} - 1.83 \cdot \frac{PI_t + DP_t}{LT_t} \\ &+ 0.285 \cdot \mathbf{1}_{NI_t<0 \text{ and } NI_{t-12}<0} - 0.521 \cdot \frac{NI_t - NI_{t-12}}{|NI_t| + |NI_{t-12}|} \end{aligned}$$

where AT corresponds to total assets, DLC to short-term debt, DLTT to long-term debt, ACT to current assets, LCT to current liabilities, LT to total liabilities, PI to pretax income, DP to depreciation and amortization, and NI to net income. We winsorize all variables except for dummy variables at the first and 99$^{\text{th}}$ percentile of their respective distribution.

**OPE - Operating Profits to Book Equity**

Following Fama and French (2015), we measure operating profits to book equity as

$$OPE_t = \frac{REVT_t - COGS_t - XSGA_t - XINT_t}{BE_t}$$

where REVT corresponds to total revenues, COGS to cost of goods sold, XSGA to selling, general and administrative expenses, XINT to interest expenses, and BE to book equity. The definition of book equity is (a) the book equity of shareholders plus (b) deferred taxes and investment tax credit (Compustat item TXDITC or TXDB + ITCB if TXDITC is unavailable) minus (c) the book value of preferred stock. Regarding (a), the book equity of shareholders is shareholders' equity (SEQ), or the sum of common equity (CEQ) and the par value of preferred stock (PSTK), or, if all previous items are unavailable, total assets (AT) minus total liabilities (LT). Regarding (b), deferred taxes (TXDITC) or investment tax credits (TXDB plus ITCB) if deferred taxes is unavailable. Regarding (c), the book value of preferred stock corresponds in the following order either to the redemption value (PSTKRV), or the liquidation value (PSTKL), or if all previous items are unavailable to the par value (PSTK). We replace missing values of items (b) and (c) with zero. Moreover, missing values in COGS, XSGA, and XINT are set to zero.

**ROA - Return on Assets**

Following Balakrishnan *et al.* (2010), we measure return on assets as quarterly income before extraordinary items (IBQ) relative to lagged quarterly total assets (ATQ), i.e.,

$$ROA_q = \frac{IBQ_q}{ATQ_{q-1}}$$

Finally, the earnings announcement date has to be after the fiscal quarter end date.

**ROE - Return on Equity**

Following Hou *et al.* (2014), we measure return on equity as quarterly income before extraordinary items (IBQ) relative to lagged book equity (BEQ)

$$ROE_q = \frac{IBQ_q}{BEQ_{q-1}}$$

Quarterly book equity (BEQ) is computed as the (a) book equity of shareholders plus (b) balance sheet deferred taxes and investment tax credit minus (c) the book value of preferred stock. Regarding (a), we measure book equity of shareholders by shareholders' equity (SEQQ), or the sum of common equity (CEQQ) and the par value of preferred stock (PSTKQ), or if all previous items are unavailable by total assets (ATQ) minus total liabilities (LTQ). Regarding (b), the balance sheet deferred taxes and investment tax credit is TXDITCQ, or TXDBQ, or zero if both are missing. Regarding (c), the book value of preferred stocks corresponds to PSTKRQ, or PSTKQ, or zero if both are unavailable. Finally, the earnings announcement date has to be after the fiscal quarter end date.

**TBI - Taxable Income to Book Income**

Following Lev and Nissim (2004), we measure taxable income to book income as pretax income (PI) scaled by net income (NI)

$$TBI_t = \frac{PI_t}{NI_t}$$

We require positive pretax and net income.

## IA-1.6. Size

**ME - Size**

Following Fama and French (1992), we measure size as the natural logarithm of the market equity. We obtain market equity data from CRSP by multiplying the shares outstanding (shrout * 1,000, as they are given in thousands) with the corresponding share price (the absolute value of altprc), i.e.,

$$ME_t = \log\left(shrout_t * 1000 * |altprc_t|\right)$$

## IA-1.7. Trading frictions

**AMI - Amihud Illiquidity Measure**

Following Amihud (2002), we measure the illiquidity from daily return-to-volume (RV), which is the absolute daily return (r) scaled by the daily dollar trading volume (product of vol and prc), i.e.,

$$RV_t = \frac{|r_d|}{prc_t * vol_t}$$

Then, the Amihud illiquidity measure corresponds to the average return to volume estimate over the last six months. We require at least 50 observations for this average and adjust the trading volume of NASDAQ stocks according to Gao and Ritter (2010).

**BETA - Beta Relative to the Market**

Following Fama and MacBeth (1973), we measure the market beta $\beta_1$ from monthly CRSP data. Specifically, we run the following time series regression over the previous five years

$$r_t^e = \alpha + \beta_1 \cdot (MKT_t - R_t^f) + u_t$$

Moreover, we require at least 24 monthly observations for the regression above.

**DTV - Dollar Trading Volume**

Following Brennan *et al.* (1998), we measure dollar trading volume from daily CRSP data as the average dollar trading volume (product of vol and prc) from month $t-6$ to month $t-1$ (with at least 50 observations). Moreover, we adjust dollar trading volume from NASDAQ according to Gao and Ritter (2010).

**ISKEW - Idiosyncratic Skewness Relative to the Fama and French (1993) three-factor model**

Following Bali *et al.* (2016), we regress daily excess returns in each month on the three Fama and French (1993) factor model (with a minimum of 15 daily observations), i.e.,

$$r_t^e = \alpha + \beta_1 \cdot (MKT_t - R_t^f) + \beta_2 \cdot SMB_t + \beta_3 \cdot HML_t + u_t$$

Then, idiosyncratic skewness is measured as the skewness of residuals $u_t$.

**IVOL - Idiosyncratic Volatility Relative to the Fama and French (1993) model**

Following Ang *et al.* (2006), we compute idiosyncratic volatility relative to the Fama and French (1993) three-factor model as the volatility of residuals from the following regression (with a minimum of 15 daily observations)

$$r_t^e = \alpha + \beta_1 \cdot (MKT_t - R_t^f) + \beta_2 \cdot SMB_t + \beta_3 \cdot HML_t + u_t$$

**MDR - Maximum Daily Return**

Following Bali *et al.* (2011), we measure the maximum daily (excess) return in each month from daily CRSP data (with at least 15 return observations).

**TUR - Share Turnover**

Following Datar *et al.* (1998), we measure the daily share turnover of each stock as the number of shares traded (vol) scaled by the number of shares outstanding (shrout), i.e.,

$$T_t = \frac{vol}{shrout}$$

Then, the share turnover for each firm in each month $t$ is the average daily share turnover over the previous six months (with at least 50 observations). Moreover, we adjust the trading volume of NASDAQ stocks according to Gao and Ritter (2010).

## IA-1.8. Valuation

### BM - Book Equity to Market Equity

Following Davis, Fama, and French (2000), we measure the book-to-market ratio as book equity (BE) from Compustat divided by market equity (ME) from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$BM_t = \frac{BE_t}{ME_t}$$

The definition of book equity is (a) the book equity of shareholders plus (b) deferred taxes and investment tax credit (Compustat item TXDITC or TXDB + ITCB if TXDITC is unavailable) minus (c) the book value of preferred stock. Regarding (a), the book equity of shareholders is shareholders' equity (SEQ), or the sum of common equity (CEQ) and the par value of preferred stock (PSTK), or, if all previous items are unavailable, total assets (AT) minus total liabilities (LT). Regarding (b), deferred taxes (TXDITC) or investment tax credits (TXDB plus ITCB) if deferred taxes is unavailable. Regarding (c), the book value of preferred stock corresponds in the following order either to the redemption value (PSTKRV), or the liquidation value (PSTKL), or if all previous items are unavailable to the par value (PSTK). We replace missing values of items (b) and (c) with zero.

### CFM - Cash Flow to Market Equity

Following Lakonishok *et al.* (1994), we measure the cash-flow-to-price ratio as income before extraordinary items (IB) plus depreciation (DP) scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$CFM_t = \frac{IB_t + DP_t}{ME_t}$$

We exclude all stocks with negative cash flows.

### DM - Debt to Market Equity

Following Bhandari (1988), we measure the debt-to-market ratio as short-term debt (DLC) plus long-term debt (DLTT) from Compustat divided by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$DM_t = \frac{DLC_t + DLTT_t}{ME_t}$$

We exclude stocks with missing DLC and DLTT observations.

**EBM - Enterprise Book Equity to Market Equity**

Following Penman *et al.* (2007), we measure enterprise book equity scaled by market equity as net debt plus book equity scaled by net debt plus market equity

$$EBM_t = \frac{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t + BE_t}{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t + ME_t}$$

where DLTT corresponds to long-term debt, DLC to current debt, PSTK to the value of preferred stock, DVPA to preferred dividends in arrears, TSTKP to preferred treasury stock, CHE to cash and short-term investments, and ME to the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD). Lastly, in this case, book equity BE is computed as common equity (CEQ) plus TSTKP minus DVPA. Missing observations in DVPA and TSTKP are set to zero. Additionally, we require that the sum of net debt and book equity, as well as the sum of net debt plus market equity, are positive.

**EM - Earnings to Market Equity**

Following Basu (1983), we measure the earnings-to-price ratio as income before extraordinary items (IB) from Compustat divided by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$EM_t = \frac{IB_t}{ME_t}$$

We exclude firms with negative earnings.

**NDM - Net Debt to Market Equity**

Following Penman *et al.* (2007), we measure net debt to price as net debt from Compustat scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$NDM_t = \frac{(DLTT_t + DLC_t + PSTK_t + DVPA_t - TSTKP_t) - CHE_t}{ME_t}$$

where DLTT corresponds to long-term debt, DLC to current debt, PSTK to the value of preferred stock, DVPA to preferred dividends in arrears, TSTKP to preferred treasury stock, CHE to cash and short-term investments. Lastly, missing observations in DVPA and TSTKP are set to zero.

**NPY - Net Payout Yield**

Following Boudoukh *et al.* (2007), we measure the net payout yield as net payouts from Compustat scaled by market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$\begin{aligned} NPY_t = &[(DVC_t + PRSTKC_t + \Delta PSTKRV_t \cdot 1_{\Delta PSTKRV<0}) \\ &- (SSTK_t - \Delta PSTKRV_t \cdot 1_{\Delta PSTKRV>0})]/ME_t \end{aligned}$$

where DVC are dividends from common stock, PRSTKC is the purchase of common and preferred stock, PSTKRV is the value of the net number of preferred stocks outstanding, and SSTK reflects the sale of common and preferred stocks. $1_{\Delta PSTKRV<0}$ is a dummy variable that has value one if the annual change in PSTKRV is negative and zero otherwise. Moreover, we exclude stocks with negative net payouts.

**OCM - Operating Cash Flow to Market Equity**

Following Desai *et al.* (2004), we measure the ratio of operating cash flows to price as operating cash flows (OC) from Compustat divided by the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$OCM_t = \frac{OC_t}{ME_t}$$

Before 1988, we measure OC as funds from operations (FOPT) minus the change in working capital (item WCAP). Thereafter, we measure OC as net cash flows from operating activities (OANCF). Moreover, we exclude firms with negative operating cash flows.

**SM - Sales to Market Equity**

Following Barbee Jr *et al.* (1996), we measure the sales-to-price ratio (SM) as sales (SALE) from Compustat divided by the market equity from CRSP (i.e., the fiscal-year end's shares outstanding, shrout, times the absolute value of their price, altprc, in million USD), i.e.,

$$SM_t = \frac{SALE_t}{ME_t}$$

**Affiliation:**

Christoph Frey
E-mail: christoph.frey@gmail.com

Christoph Scheuch
E-mail: christoph@tidy-intelligence.com

Stefan Voigt
E-mail: stefan.voigt@econ.ku.dk

Patrick Weiss
E-mail: patrickw@ru.is

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd